A Provable Softmax Reputation-Based Protocol for Permissioned Blockchains

Hongyin Chen, Zhaohua Chen, Yukun Cheng, Xiaotie Deng, *Fellow, IEEE*, Wenhan Huang, Jichen Li, Hongyi Ling, and Mengqian Zhang

Abstract—We consider a hierarchical structure of a permissioned blockchain with three types of participant: providers, collectors, and governors. Providers forward transactions to collectors; collectors upload received transactions to governors after verifying and labeling them; and governors validate a portion of the labeled transactions they receive, pack valid transactions into a block, and append the block to the ledger. This model has various fields of application including data collection from the Internet-of-Things and second-hand markets. Our main contribution is to propose a reputation-based protocol to help governors evaluate the reliability of collectors. Specifically, given a transaction, each governor runs a softmax-based function to calculate a probability for each collector that sent and labeled this transaction. The probabilities, calculated using collectors' reputations as inputs, represent the likelihood of the lead governor selecting the labeled transaction from collectors to consider for further validation. After the lead governor verifies a transaction, all collectors' reputations are updated in line with the agreement of their labeling and the validity of the transaction as found by the lead governor. We show, both theoretically and empirically, that our protocol can significantly reduce governors' verification workloads while maintaining firm liveness and high incentives.

Index Terms—Permissioned blockchain, transaction verification, reputation algorithm, hierarchical structure

1 Introduction

Blockchains' increasing popularity has brought them widespread attention. Permissioned blockchains—which require participants to be identified—are of great significance to many situations using blockchains. An impressive example is Hyperledger [1], which has been widely applied in various fields including finance, manufacturing, and supply chains.

Permissioned blockchains are far from being used to their full potential. This work demonstrates their use in solving an essential decision-making problem related to the Internet-of-Things (IoT). A key issue facing the IoT is to collect of useful data from the millions of front-end sensors to analyze users' behaviors and habits. Due to limitations of processing capacity and network bandwidth, low-cost sensors cannot feasibly upload filtered data directly to cloud nodes. Instead, a layer of gateway nodes has to aggregate data from sensors and send the processed data to cloud nodes.

The system largely depends on the gateway nodes: their unreliability would break the whole system. Malicious gateway nodes deliberately misjudging data would affect subsequent decisions by the cloud nodes. This is a long unresolved problem facing the IoT. Section 6 describes similar problems that face second-hand markets.

A permissioned blockchain can handle this difficulty. The decentralization of a blockchain system allows gateway nodes to join the system simultaneously and distributedly. The use of permissions ensures that nodes in the blockchain system are identified, which meets the IoT's requirement for gateway nodes' information to be specified. A crucial feature is that, as blockchains are tamperproof, the behavior of gateway nodes can never be misclaimed. Therefore, the gateway nodes can be further assessed to help reduce the long-term costs due to dishonest nodes.

This work considers a hierarchical permissioned blockchain model suitable for decision making in the IoT. The model's three kinds of node are *providers*, *collectors*, and *governors*. Providers resemble sensors and offer original transactions to collectors. Collectors, resembling gateway nodes, verify the transactions from providers and send labeled transactions to governors. Governors, analogous to cloud nodes, process transactions, pack them into blocks, and update the ledger. In each round, a leading governor is in charge of processing transactions and appending a block to the ledger. As all participants in a permissioned blockchain are identified, each collector's trustfulness in the current task can be derived from its long-term behavior. This facilitates

- *Hongyin Chen, Zhaohua Chen, Xiaotie Deng, and Jichen Li are with the Center on Frontiers of Computing Studies, Computer Science Department, Peking University, Beijing 100871, China. E-mail: {chenhongyin, chenzhaohua, xiaotie, 2001111325}@pku.edu.cn.*
- *Yukun Cheng is with the School of Business, Suzhou University of Science and Technology, Suzhou, Jiangsu 215009, China. E-mail: ykcheng@amss.ac.cn.*
- *Wenhan Huang and Mengqian Zhang are with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China. E-mail: {rowdark, mengqian}@sjtu.edu.cn.*
- *Hongyi Ling is with the Department of Computer Science, ETH Zurich, 8092 Zurich, Switzerland. E-mail: holing@student.ethz.ch.*

Manuscript received 31 May 2021; revised 14 Nov. 2021; accepted 20 Nov. 2021. Date of publication 24 Nov. 2021; date of current version 8 Mar. 2023.
This work was partially supported by the National Major Science and Technology Projects of China-"New Generation Artificial Intelligence" under Grant 2018AAA0100901, the National Natural Science Foundation of China under Grant 11871366, and Qing Lan Project.
(Corresponding authors: Yukun Cheng and Xiaotie Deng.)
Recommended for acceptance by Y. Yang.
Digital Object Identifier no. 10.1109/TCC.2021.3130244

the development of a reputation-based protocol that evaluates each collector's reputation according to its past behavior, thus giving a measure of its reliability.

1.1 Our Contributions

Our main contribution here is the design of a *softmax reputation-based protocol* to help governors evaluate the behavior of collectors and reduce their costs of verification during decision-making. Specifically, the governors maintain a local reputation vector over all collectors that reflects their reliability. As with many real-world cases (e.g., in the IoT), multiple collectors *overlappingly* gather transactions from providers. In other words, a transaction may be processed by multiple collectors in parallel before being considered by a governor. Therefore, different collectors could variously label the same raw transaction. Motivated by this possible scenario, we suppose here that any collector can label a transaction as either valid (+1) or invalid (-1). This captures the possibility of faults in the low-level sensors of the IoT.

A leader, elected from all of the governors, is responsible for deciding whether to verify each transaction. First, when this governor receives a transaction with binary labels from multiple collectors, it selects one of the collectors with a probability calculated by the softmax function using the collectors' reputations as inputs. The governor then decides whether to verify the transaction according to the sampled label from the selected collector. If the leader decides not to verify it, this transaction is neglected and added to $UncheckedList$. Otherwise, this transaction will be added to $TXList$ (if found valid) or $InvalidList$ (if invalid). Only valid transactions (i.e., in $TXList$) will be included in a block on the ledger. Governors then compare the verification result with each collector's labeling of the transaction, and those collectors who wrongly labeled will face a decrease of -1 in reputation.

Theoretical analysis demonstrates that our softmax reputation-based protocol can significantly improve efficiency. Specifically, for any governor g and provider p, the number of invalid transactions provided by p (out of T_{total} transactions sent in total by p) and further verified by g is asymptotically $O(\sqrt{T_{total}})$, as long as at least one honest collector is linked with p. Verifying an invalid transaction is deemed a loss for the governor. Practically, verifying invalid transactions is a huge waste of resources with no gain. Therefore, our reputation mechanism limits this loss to a satisfactory level.

Regarding liveness, we illustrate that our protocol allows any valid transaction to be verified and included in the blockchain within constant rounds in expectation, given that a constant fraction of collectors linked by any provider are honest. Here we assume that an honest collector always resubmits a valid transaction if the transaction is neglected (i.e., appended to $UncheckedList$).

A collector's reputation reflects its reliability. We can thus explicitly motivate collectors to work honestly by distributing rewards according to their reputation. We prove that under our system, any collector who mislabels a transaction will see a decrease in reward in expectation.

Empirically, our simulation results further corroborate that our protocol brings high efficiency, firm liveness, and promising incentives.

Finally, we introduce some applications of our protocol to real-life scenarios: the IoT and second-hand markets. It elegantly resolves severe issues around efficiency, liveness, and incentives that concern the design of a reputation-based protocol for permissioned blockchains.

1.2 Related Work

Nakamoto first proposed in 2008 the blockchain as a distributed tamperproof ledger for recording numerous digital transactions [2]. Such a public data-ledger is maintained by reaching a consensus among a number of nodes in a peer-to-peer network. These transactions are packed into blocks and then appended to the ledger after verification. Based on the access control scheme [3], blockchain systems can be either permissionless or permissioned. Permissionless blockchains are generally called public blockchains. Examples include Bitcoin [2], Ethereum [4], and Litecoin [5]. Anyone can join these as a validator without authorization. Several consensus protocols, such as *proof-of-work* (PoW) and *proof-of-stake* (PoS), are applied in permissionless blockchains to reach a consensus based on participants' computing power or wealth. In contrast, permissioned blockchains, including private and consortium blockchains, require all participants to be identified. Unlike permissionless blockchains, these blockchains are operated only by known entities. For example, a *private blockchain* has only one trusted entity in charge [6]. A *consortium blockchain* has members of a consortium or business managing a *permissioned blockchain* network. permissioned blockchain networks usually reach the consensus among only a small group of authenticated nodes, and some of them adopt practical Byzantine fault tolerance (PBFT) protocols [7] to reach a consensus. Permissioned blockchains allow each participant's trustfulness in a current task to be tied to its historic behavior. This can be used to build a reputation-based protocol, whereby a participant's reputation is evaluated according to its past behavior and represents a measurement of reliability.

Reputation in *peer-to-peer* networks has been studied for decades. As the name suggests, reputation represents a peer's reliability and is evaluated according to past behavior. Reputation has been studied extensively in contexts such as systems design [8], [9], security issues [10], [11], and privacy protection [12], [13]. The recent emergence and development of blockchain technology has added a new aspect to research on reputation. The existing works focus mainly on the following two points.

First, blockchains' immutability and decentralization lend them to recording and sharing information in a reputation system [14], [15], [16]. For example, transactional histories stored on a permissioned blockchain represent evidence that can be evaluated in a distributed manner to establish the reputations of all registered participants [17]. Instead of adopting traditional consensus protocols like PoW or PoS when generating blocks, a new protocol, called *proof-of-reputation* (PoR), can be used [17]. To be specific, the PoR protocol assigns the most-reputable node involved in the current block to be the leader of the current round. Participants all believe that more-reputable nodes provide better services. By fully utilizing reputation, the PoR protocol performs efficiently in the designed system.

Second, the concept of reputation can be used as an incentive in blockchain applications. Our work falls into this

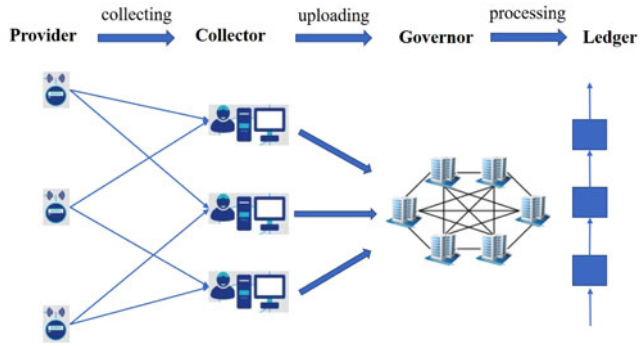


Fig. 1. Hierarchical model of the system.

research area. A reputation-based framework has been developed to avoid dishonest mining strategies in blockchain systems that use PoW as their consensus protocol [18]. In the framework, each node has a public reputation value, representing how well it has performed previously in the system. When the pool manager sends invitations to miners to form a mining pool for the proof-of-work computation, a node's probability of being invited is related to its reputation value. A more-reputable node has a better chance of being invited to mining pools; it thus gains more revenue. As the public reputation system is sustained over time, miners are incentivized to behave honestly to maximize their long-term utility. However, the prior publication only lists several possible influence factors and qualitatively analyzes the update of reputation values [18]; it does not give concrete forms of the update and probability functions. A protocol called CycLedger for a sharding-based blockchain introduces the concept of reputation to provide nodes with a sufficient incentive to enter the system [19]. Nodes in the system are required to give opinions on the validity of requested transactions. Reputation is computed according to the cosine similarity between the given opinions and the consensus results. A node's reputation thus reflects the honest computational resources it has contributed to the system. Assigning nodes with more computational resources to high-workload positions further enhances efficiency. As a higher reputation brings more revenue, the protocol attracts nodes to participate actively and honestly in the system.

In the context of previous works on reputations, our softmax reputation-based protocol is the first to use reputation as an incentive during transaction transmission in permissioned blockchains. Our protocol motivates collectors to label transactions correctly, and thus reduce the governors' verification costs.

The rest of the paper is arranged as follows. Sections 2 and 3 formulate the model and introduce the softmax reputation-based protocol. Theoretical analysis of our protocol in Section 4 is followed by numerical experiments in Section 5 that demonstrate its effectiveness. Section 6 considers real-world application of the softmax reputation-based protocol. The last section concludes the work.

2 Model

2.1 Types of Node and Transaction

This work considers a hierarchical model comprising three types of node: providers, collectors, and governors (Fig. 1).

- *Providers* provide original transactions to collectors. Although the correctness of transactions is not guaranteed, providers sign transactions and add a *timestamp* so that no collector can forge a transaction. The set of providers is denoted by P with $|P| = l$.
- *Collectors* receive and verify transactions from providers and upload them to governors. Specifically, after receiving a signed transaction from a provider, a collector checks and labels it either $+1$ or -1. A $+1$ label implies that the transaction is considered *valid*; -1 represents invalidity. The collector then submits the labeled transaction together with its own signature to governors. The set of all n collectors in the system is denoted by C: its subset $C_i = \{c_i(1), \ldots, c_i(u_i)\}$ contains the u_i collectors connected to provider $p_i \in P$.
- *Governors* generate blocks. In each round, a governor elected as the leader is responsible for processing transactions and proposing a block. For each original transaction tx, the leader may receive it with different labels from several collectors. It must then decide whether to verify tx; it does so by considering the reputations of the sending collectors. Finally, the leader generates a block containing valid transactions and the Merkle tree root of other transactions. Let G be the governor set and there are $|G| = m$ governors in the system. We suppose that the connections between providers and collectors are known to all governors. All governors are assumed to always behave honestly in reaching consensus. This assumption captures the practical scenario of real-life governors mainly being large companies that would face a potentially huge reputational loss if found cheating.

In many practical scenarios, the cost of verifying a transaction can be high, as a governor must undertake the substantial task of directly investigating the provider. For a governor to verify a transaction as invalid is a waste of its resources. Therefore, we define the *loss* related to provider p as the total number of invalid transactions provided by p that are checked by governors.

As mentioned, this work considers two types of transaction: either unlabeled or labeled.

- *tx*: An unlabeled transaction tx is one as originally sent by a provider. It includes the signature of the provider.
- *TX*: A labeled transaction TX comprises an original transaction tx, a $label$ given by collector c, and $sig_c(tx, label)$, which is a digital signature of $(tx, label)$ generated by c:

$$TX = (tx, label, sig_c(tx, label)).$$

Generally, both are called "transactions," but they are explicitly distinguished using tx or TX in the following algorithms.

2.2 Network Assumptions

Our protocol is implemented in a *permissioned blockchain*. The identities of the nodes in the network are all maintained by an identity manager, which also records nodes' roles in

TABLE 1
Main Notation

$P = \{p_1, p_2, \cdots, p_l\}$	the set of providers; $\|P\| = l$
$C = \{c_1, c_2, \cdots, c_n\}$	the set of collectors; $\|C\| = n$
$G = \{g_1, g_2, \cdots, g_m\}$	the set of governors; $\|G\| = m$
$C_i = \{c_i(1), \cdots, c_i(u_i)\}$	the subset of collectors connected with provider p_i; $\|C_i\| = u_i$
$r_i(k)$	the reputation of provider p_i's k^{th} connected collector (*i.e.*, $c_i(k)$), which is maintained by governors
tx	the original transaction
TX	the labeled transaction
$TXList$	the list of valid transactions
$InvalidList$	the list of invalid transactions
$UncheckedList$	the list of unverified transactions
cnt_i	the number of transactions from provider p_i that have been verified by governors in the slot
T_i	the number of transactions from provider p_i that need to be processed in the slot before profit allotting
F_i	the total profit from transmitting transactions of provider p_i in the slot
$V_i(k)$	the collector $c_i(k)$'s revenue for transmitting transactions from provider p_i in the slot

the blockchain system. It provides nodes with credentials that are used for authentication and authorization. By default, the identity manager contains all standard public-key infrastructure methods and plays the role of a certificate authority. Unless specified otherwise, all interactions within the network are authenticated via digital signatures and the identity manager offers each node this scheme.

As the underlying system is permissioned, we assume it to be synchronous. That is, there is a known upper bound on processing delays and message transmission delays. Meanwhile, each node is equipped with a local physical clock, and there is an upper bound on the rate at which any local clock deviates from a global real-time clock [20].

2.3 Notation

Table 1 summarizes the main notation. We emphasize that reputation here reflects the governors' evaluation of a collector's reliability. The example in Fig. 2 shows provider p_i sending an original transaction tx to all of its connected collectors. The collectors verify it, label it, and upload it to all governors. For collector $c_i(k)$ connected to p_i, $r_i(k)$ represents the reliability of the collector's verification result. Then $\{r_i(k)\}_{k\in(1,u_i)}$ represents a measurement of the governors' trust in the collectors that may upload the same original transaction from provider p_i but with different labels.

The following functions are considered subsequently.

- $broadcast_{prvd}(tx)$. A provider uses this to send an original transaction tx to its linked collectors.
- $broadcast_{clct}(TX)$. A collector runs this to disseminate a labeled transaction TX to all governors.

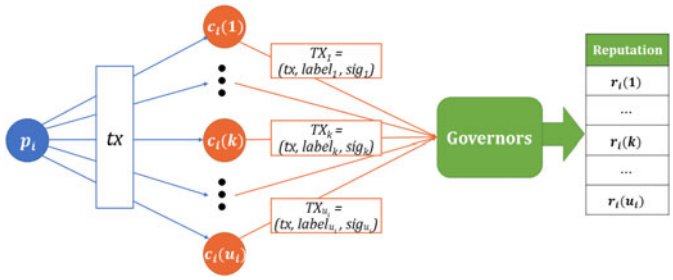


Fig. 2. An example demonstrating the notation.

- $broadcast_{gvrn}(msg)$. Each governor can execute this operation to broadcast message msg to all other governors.
- $broadcast_{all}(msg)$. Each node can use this to broadcast message msg to all linked nodes.
- $validate_{clct}(tx)$. A collector runs this to check whether transaction tx is valid. It returns $true$ if tx is valid, and $false$ otherwise. The direct connections between collectors and providers ensure that this function is not costly.
- $parse(TX)$. This returns $(tx, label)$ for labeled transaction TX, where tx is the original transaction and $label$ is $+1$ or -1. The $label$ assigned by a collector may not match the result of $validate_{clct}(tx)$; e.g., a collector may find $validate_{clct}(tx)$ to give $true$, yet label tx as -1.
- $validate_{gvrn}(tx)$. A governor can run this to check whether a parsed transaction tx is valid. It returns $true$ if tx is valid, and $false$ otherwise. As governors are far from the providers, this function runs at great cost.

Note that digital signatures cannot be forged, and thus for simplicity we ignore the process of verifying them.

2.4 Protocol Overview

This paper proposes a protocol to collect, verify, and pack transactions in a permissioned blockchain by introducing reputation as a measure of reliability to encourage collectors to correctly label the transactions. A major aim is minimizing the number of invalid transactions verified by governors, thus minimizing the verification loss of the whole system. The protocol is implemented in multiple *rounds*, each containing three phases.

- *Collecting.* An original transaction tx is offered by a single provider to several collectors.
- *Uploading.* Collectors verify the received original transactions and label them. These labeled transactions, denoted by TX, are then uploaded to governors.
- *Processing.*In each round, a leader is elected by all governors via PoS. An original transaction tx might be contained in several labeled transactions uploaded by the collectors. The leader chooses a collector according to its reputation, and decides whether to verify tx according to the label this collector assigns to tx. If the leader decides not to verify, then tx is neglected and added to $UncheckedList$. Otherwise, the leader verifies tx and adds it to $InvalidList$ or $TXList$ if it is found to be $invalid$ or

$valid$, respectively. After verification, the leader could make judgment on the collectors by comparing their labels of tx and its own verification result. It uses this to update the collectors' reputations. At the end of each round, the leader proposes a block, and all governors reach a consensus on this block as well as the updated reputations of the collectors.

3 Protocol Design

3.1 Collecting Phase

Providers first offer original transactions to collectors. Specifically, each provider $p_i \in P$ signs the transaction and adds a timestamp t to constitute tx. It then invokes $broadcast_{prvd}(tx)$ to send tx to all u_i collectors connected to provider p_i.

3.2 Uploading Phase

Collectors process the transactions from connected providers and then verify and upload them to governors. Consider collector $c \in C$ receiving transaction tx from provider p: It runs $validate_{clct}(tx)$ to check whether tx is valid and obtains a result of either $true$ or $false$. Collector c then attaches a label of either $+1$ or -1 to tx. The $label$ can be inconsistent with the result of $validate_{clct}(tx)$. For example, a dishonest collector c may validate tx as $true$ but label it -1. After c processes, verifies, and labels tx, it signs $(tx, label)$ to generate labeled transaction TX, which it broadcasts to all governors. Algorithm 1 describes uploading by an honest collector.

Algorithm 1. Transaction Uploading

```
1: procedure TRANSACTIONUPLOAD                    ▷For collector c
2:     upon ⟨ deliver_prvd | p, tx ⟩ do
3:         if validate_clct(tx) = true then
4:             label ← +1
5:         else
6:             label ← −1
7:         TX ← (tx, label, sig_c(tx, label))
8:         invoke broadcast_clct(TX)
```

3.3 Processing Phase

A leader elected by the PoS protocol from all governors processes the transactions sent by the collectors and proposes a new block. The leader may receive several copies of an original transaction tx with potentially differing binary labels from different collectors. The leader decides whether to verify tx according to the reputations of the collectors forwarding it. If the leader decides not to verify transaction tx, it is added to $UncheckedList$; otherwise, the transaction is appended to $TXList$ if found "valid" or $InvalidList$ if found "invalid". The governors then update the collectors' reputations. At the end of each round, the leader generates a block. When a consensus on the block is reached by all governors, the newly generated block is appended to the ledger, and a new round starts. More specifically, each round contains the following five steps.

3.3.1 Transaction Screening

Transaction screening involves the leader receiving transactions and deciding whether to verify each one individually. Each transaction is then appended to one of the lists $UncheckedList$ (if not checked), $TXList$ (if checked and verified), or $InvalidList$ (if checked and discarded). As mentioned above, we assume transactions to have only correct signatures, as digital signatures cannot be forged in a permissioned blockchain.

Consider provider p_i proposing transaction tx. It will broadcast tx to all its connected collectors, $c_i(k) \in C_i$. The leader starts a timer upon receiving the first copy of tx with an attached label. Within a time of Δ, the leader may receive another $x - 1$ copies of tx, each with a label from the collectors in C_i. Here, $x \leq u_i$, as some collectors may simply discard tx or may not finish verifying it in time.

Algorithm 2. Transaction Screening

```
1: procedure TRANSACTIONSCREEN                    ▷For governor g_ld
2:     upon ⟨ deliver_clct | c, TX ⟩ do
3:         (tx, label) ← parse(TX)
4:         if received[tx] = ∅ then
5:             startTime(tx, Δ)                   ▷start timing
6:         if (c, −label) ∉ received[tx] then
7:             received[tx] ← received[tx] ∪ {(c, label)}
8:
9:     upon endTime(tx) do                        ▷time up
10:        for all collectors c_i(k) ∈ C_i do
11:            Prob(k) ← exp(ηr_i(k)) / Σ_{c_i(j)∈C_i} exp(ηr_i(j))   ▷softmax
12:        draw a collector c_i(k) ∈ C_i w.p. Prob(k)
13:        if (c_i(k), +1) ∈ received[tx] then
14:            validbit ← validate_gvrn(tx)
15:            cnt_i ← cnt_i + 1
16:            msg ← (tx, validbit, received[tx], cnt_i)
17:            invoke broadcast_gvrn(msg)
18:            if validbit = true then
19:                TXList ← TXList ∪ {tx}
20:                invoke reputationUpdate(tx, +1)
21:        else
22:            InvalidList ← InvalidList ∪ {tx}
23:            invoke reputationUpdate(tx, −1)
24:    else
25:        UncheckedList ← UncheckedList ∪ {tx}
```

The leader chooses a collector $c_i(k) \in C_i$ with a probability which is computed from the softmax function and proportional to $\exp(\eta r_i(k))$. Here, $r_i(k)$ is the reputation of collector $c_i(k)$ with respect to its link with p_i, and $\eta > 0$ is a predetermined parameter. Consider the chosen collector, $c_i(k)$. If $c_i(k)$ fails to send a copy of tx (either intentionally or unintentionally), it is assumed to have sent a label of -1. If the label of tx by $c_i(k)$ is -1, then the leader does not check it and appends it to $UncheckedList$. Otherwise, the leader verifies tx, and uses the result (valid or invalid) to append the transaction to either $TXList$ or $InvalidList$, respectively.

Algorithm 2 illustrates transaction screening. To avoid being cheated, the leader checks if collector c has mislabeled tx (Line 6). Here, $received[tx]$ contains all received verification opinions on this tx. In lines 14–16, the leader increases the counter cnt_i by 1 and broadcasts $(tx, validbit, received[tx], cnt_i)$ to other governors after verifying tx. Here, cnt_i records the number of transactions from provider p_i that have been verified and will be used in the following

profit allotting step (see Section 3.4). This broadcasting step informs how the collectors labeled tx and whether tx is valid. Then, all governors trigger $reputationUpdate$ to synchronize the reputation (detailed in Section 3.3.2).

Algorithm 2 neglects two details. First, it does not matter which governor validates tx, as governors do not tolerate wrong transactions. Thus, the leader can ask any governor to trigger $validate_{gvrn}(\cdot)$ and get the result. Second, the waiting time Δ may not be contained in a round, thus collectors will send TX to all governors including the leader, and other governors will also maintain $received[tx]$.

3.3.2 Reputation Updating

Collectors' reputations are updated after the leader verifies transaction tx. All governors use the validation result received from the current leader to update the reputations of the collectors that labeled tx.

For transaction tx provided by p_i and validated by the leader, the reputations of collectors that label tx correctly will remain unchanged. The other collectors in C_i that reported the opposite label or failed to report will receive a loss of one unit in their reputations. If transaction tx is not verified, then the leader will not update any reputations. The idea of this design resembles the *Multiplicative Weights Update Method* [21]. Section 4 gives more-detailed analysis. Algorithm 3 describes this reputation updating process.

Algorithm 3. Reputation Updating

```
1: procedure REPUTATIONUPDATE(tx, status)  ▷For governor g
2:   if status = +1 then
3:     for c_i(k) ∈ C_i do
4:       if (c_i(k), +1) ∉ received[tx] then
5:         r_i(k) ← r_i(k) − 1
6:   if status = −1 then
7:     for c_i(k) ∈ C_i do
8:       if (c_i(k), +1) ∈ received[tx] then
9:         r_i(k) ← r_i(k) − 1
```

3.3.3 Block Proposing

The leader g_{leader} generates a block,

$$B = (h,\ sn,\ g_{leader},\ TXList = \{tx_1, \ldots, tx_b\},\ MT),$$

and appends it to the ledger. Note that any block in the ledger is tamper-proof insofar as it contains the hash value h of the previous block. Moreover, each block has a serial number sn that is one greater than that of the preceding block. Here, h and sn are easily calculated given the former block. In addition, the block also contains the list of verified valid transactions $TXList$, which is generated during *transaction screening* (when $InvalidList$ and $UncheckedList$ are also generated). Here, the subscript b is the number of valid transactions in the block, whose upper-bound is b_{limit} to restrict the size of any one block. The leader also computes the Merkle root (MT) of $\{InvalidList,\ UncheckedList\}$, and puts it in the block.

After generating a new block B, the leader broadcasts it to all other nodes by invoking $broadcast_{all}(B)$. The leader also triggers $broadcast_{all}(\{InvalidList, UncheckedList\})$ to inform the nodes of the invalid or unchecked transactions. Upon receiving the information, a provider can ascertain whether its own valid transactions have been checked and then decide whether to resend them.

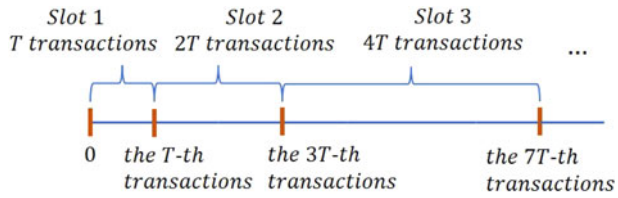


Fig. 3. Provider p_i's whole process of transmitting transactions.

3.3.4 Leader Election

The final processing step is the election of a new leader for the next round from among all of the governors. Considering the permissioned environment, the leader is elected by a PoS protocol.

Specifically, suppose governor $g \in G$ owns several units of stake. In practice, the stake may be cash or any reliable asset. In round r, each governor calculates a random string s with proof π via a *verifiable random function* (VRF) [22] for each unit of stake. Then each governor $g \in G$ disseminates its $\{\langle s, \pi\rangle\}_g$ to all other governors by invoking $broadcast_{gvrn}(\cdot)$. After receiving all of the random strings and corresponding proofs from the other governors, a governor first validates the proof to check whether the string value is correct. For all correct strings, the owner of the stake unit with the minimal string value becomes the leader of this round.

This VRF scheme gives each governor a chance to be chosen as the leader, with a probability proportional to its stake. Recall that governors in the system are honest. As stakeholders, there is no motivation for them to damage the chain. In this sense, the VRF scheme ensures the output string is pseudorandom and unpredictable.

3.4 Profit Allotting

Besides the above mentioned three phases our protocol contains, it also can motivate collectors to behave truthfully by allowing governors to allot profit to collectors based on their reputations. In detail, for each provider p_i, its whole process of transmitting transactions is divided into several slots, in each of which p_i uploads several transactions (shown in Fig. 3). Thus, governors maintains two counters: cnt_i and T_i, $i = 1, \ldots, l$. The former records the number of transactions from provider p_i that have been verified in each slot. The second counter T_i is the total number of transactions from provider p_i to be dealt with in the current slot. The slot ends when cnt_i reaches the current value of T_i and a new slot begins at the same time. T_i is initially set to a predetermined integer $T > 0$ and then doubled every time a new slot begins. Recall that p_i has u_i collector neighbors in set $C_i = \{c_i(1), \ldots, c_i(u_i)\}$, and each has a certain reputation $r_i(k)$ for $\forall 1 \le k \le u_i$.

As the leader deals with all transactions one-by-one and updates the reputations $\{r_i(k)\}$ of all collectors correspondingly. Let $r_i^{(cnt_i+1)}(k)$ denote the expected reputation of $c(k)$ after dealing with the cnt_i-th transaction in one slot. Once

the current slot ends, a constant proportion of the profit gained by executing these transactions is allotted to the collectors in C_i according to their reputations. If F_i is the profit coming from transmitting transactions of p_i in current slot, at the end of this slot, each collector $c_i(k)$ shall obtain its revenue, $V_i(k)$, which is given as follows,

$$V_i(k) = \frac{\exp(\eta r_i^{(T_i+1)}(k))}{\sum_{j=1}^{u_i} \exp(\eta r_i^{(T_i+1)}(j))} \cdot F_i.$$

After allocation of the profit at the end of each slot, the reputations of the collectors connected to provider p_i are reset to 0, i.e., each element of $\{r_i(k)\}_{k\in(1,u_i)}$ is 0. The value of T_i is then doubled. As cnt_i is also broadcast among governors, the collectors' reputations are easily synchronized by governors even after they are reset.

4 Analysis

This section first presents the safety features that our protocol provides. We also demonstrate that our softmax-based reputation protocol increases efficiency, ensures liveness, and provides suitable incentives. Specifically, we consider the spindle-shaped case with only one governor g, one provider p, and u collectors connected to both p and g. This case can be directly generalized to a broader range of cases, as the whole process—collecting, verifying, packing transactions, and reputation updating—is independent of the governor–provider pairs.

4.1 Safety Guarantees

Our protocol is implemented on a permissioned blockchain system that largely resembles Hyperledger Fabric [1] in the consensus layer. Therefore, it has the following properties.

- *Agreement*: For any two blocks B and B' retrieved with the same serial number s, $B = B'$;
- *Chain Integrity*: For any two blocks B and B', if B is retrieved with serial number s while B' is with serial number $s+1$, then the hash value h' contained in B' satisfies $h' = H(B)$, where H is a public collision-resistant hash function (CRHF).
- *No Skipping*: Once a block with the serial number s is retrieved, blocks with all previous serial numbers (i.e., $1, \cdots, s-1$) are already retrieved.
- *Almost No Creation*: If an *honest* node perceives tx when retrieving a block B, then excluding the negligible probability of λ (where λ is the security parameter of the protocol), tx has been broadcast previously via both $broadcast_{prvd}(\cdot)$ and $broad-cast_{clct}(\cdot)$.

As governors are completely trustworthy when reaching a consensus, blocks are proposed safely. Thus, every governor adopts the same block in each round, which derives the properties of *Agreement*, *Chain Integrity*, and *No Skipping* in our protocol. At the same time, any collector cannot forge transactions without a provider's public key except with the negligible probability of the security parameter λ. Our protocol thus establishes the property of *Almost No Creation*.

Here we elaborate on the consensus-reaching process not imposing a great burden on the system. To reach a consensus on a block that contains transactions offered by governors, the communication complexity is $O(b_{limit}m)$. Here, b_{limit} is the maximum number of transactions in a block, and m is the number of governors. To generate a stake-transform block, governors need to broadcast a transaction message to each other, which entails a communication complexity of $O(m^2)$. Nevertheless, in any real-life implementation of our permissioned blockchain scheme, there are far fewer governors (e.g., about 10) than providers and collectors (e.g., more than 100). Therefore, the complexity of consensus-reaching is not a bottleneck in our system.

4.2 Efficiency Analysis

To measure the efficiency of the mechanism, we first define the loss for a governor when verifying transactions.

Definition 1. *For a set of transactions verified by a governor g, the* loss *for g when verifying these transactions is defined as the number of invalid transactions.*

The definition of loss represents that a governor wastes its resources when verifying an invalid transaction and this should be avoided whenever possible.

We now give the main theorem describing the efficiency of our softmax-based reputation mechanism.

Theorem 1. *Consider a provider p, a governor g, and a group of u collectors simultaneously connected with g and p. Suppose T transactions have been offered by p and verified by g. Let $\mathcal{L}_T$ be the accumulated expected verification loss of these transactions for governor g, and $\mathcal{S}_T^{min}$ be the accumulated verification loss for the best-behaving collector on these transactions. For any parameter $\eta > 0$, we have*

$$\mathcal{L}_T - \mathcal{S}_T^{min} \leq \frac{\ln u}{\eta} + \frac{\eta T}{2}. \tag{1}$$

Proof. For simplicity, let $r(k)$ denote the reputation of collector $c(k)$ connected to provider p (where $k = 1, \ldots, u$). Our result for the case of one governor and one provider is an extension of the result for the Multiplicative Weights Update method [21]. We follow the potential method to prove (1).

The governor deals with all T transactions one-by-one and updates the reputations $\{r(k)\}$ of all collectors correspondingly. Let $r^{(t)}(k)$ denote the expected reputation of $c(k)$ before dealing with the t-th transaction. Then the probability of g choosing specific collector $c(k)$ for the t-th transaction is $Prob^{(t)}(k) = \frac{\exp(\eta r^{(t)}(k))}{\sum_{j=1}^{u} \exp(\eta r^{(t)}(j))}$. The reputation change of $c(k)$ is $w^{(t)}(k) = r^{(t+1)}(k) - r^{(t)}(k)$. Note that for the t-th transaction, if g chooses $c(k)$, one of the following events must happen.

- If collector $c(k)$ labels the t-th transaction as -1, then governor g decides not to verify it. So $w^{(t)}(j) = 0$ and thus $r^{(t+1)}(j) = r^{(t)}(j)$, for any $j = 1, \ldots, u$;
- If collector $c(k)$ labels the t-th transaction as $+1$, then the governor verifies the transaction.
  - If the verification result is "valid", then $w^{(t)}(j) = 0$ for each collector $c(j)$ that labeled

the t-th transaction as $+1$, and $w^{(t)}(j) = -1$ for any collector $c(j)$ that labeled the t-th transaction as -1 or failed to report this transaction;
- If the verification result is "invalid", then $w^{(t)}(j) = -1$ for all collectors that labeled the t-th transaction as $+1$, and $w^{(t)}(j) = 0$ for others.

Let us define the potential function as $\Phi(t) = \frac{1}{\eta}\ln\sum_{k=1}^{u}\exp\big(\eta r^{(t)}(k)\big)$. Therefore,

$$\begin{aligned}\Phi(t+1)-\Phi(t) &= \frac{1}{\eta}\ln\frac{\sum_{k=1}^{u}\exp\big(\eta r^{(t+1)}(k)\big)}{\sum_{j=1}^{u}\exp(\eta r^{(t)}(j))}\\ &= \frac{1}{\eta}\ln\sum_{k=1}^{u}\frac{\exp\big(\eta(r^{(t)}(k)+w^{(t)}(k))\big)}{\sum_{j=1}^{u}\exp(\eta r^{(t)}(j))}\\ &= \frac{1}{\eta}\ln\sum_{k=1}^{u}Prob^{(t)}(k)\exp\Big(\eta w^{(t)}(k)\Big)\\ &\le \frac{1}{\eta}\ln\sum_{k=1}^{u}Prob^{(t)}(k)\left(1+\eta w^{(t)}(k)+\frac{\big(\eta w^{(t)}(k)\big)^2}{2}\right)\end{aligned} \tag{2}$$

$$\begin{aligned}&= \frac{1}{\eta}\ln\left(1+\sum_{k=1}^{u}Prob^{(t)}(k)\left(\eta w^{(t)}(k)+\frac{\eta^2\big(w^{(t)}(k)\big)^2}{2}\right)\right)\\ &\le \frac{1}{\eta}\sum_{k=1}^{u}Prob^{(t)}(k)\left(\eta w^{(t)}(k)+\frac{\eta^2\big(w^{(t)}(k)\big)^2}{2}\right)\end{aligned} \tag{3}$$

$$\begin{aligned}&= \sum_{k=1}^{u}Prob^{(t)}(k)w^{(t)}(k)+\frac{\eta\sum_{k=1}^{u}Prob^{(t)}(k)\big(w^{(t)}(k)\big)^2}{2}\\ &= -L(t)+\frac{\eta\sum_{k=1}^{u}Prob^{(t)}(k)\big(w^{(t)}(k)\big)^2}{2}\le -L(t)+\frac{\eta}{2},\end{aligned} \tag{4}$$

where $L(t)$ is the expected verification loss for the tth transaction; (2) holds as $e^x \le 1+x+\frac{x^2}{2}$ for $x\le 0$, and $w^{(t)}(k)\le 0$; (3) holds because $\ln(1+x)\le x$ for $x\ge -1$; and (4) holds as $\big|w^{(t)}(k)\big|\le 1$ and $\sum_{k=1}^{u}Prob^{(t)}(k)=1$. Let us sum (4) from $t=1$ to T, and notice that $\Phi(1)=\frac{\log u}{\eta}$. Then, we get

$$\Phi(T+1)-\frac{\ln u}{\eta}\le -\sum_{t=1}^{T}L(t)+\frac{\eta T}{2}=-\mathcal{L}_T+\frac{\eta T}{2},$$

which is equivalent to

$$\mathcal{L}_T+\Phi(T+1)\le\frac{\ln u}{\eta}+\frac{\eta T}{2}. \tag{5}$$

Finally, by the definition of $\Phi(T)$, we have

$$\begin{aligned}\Phi(T+1) &= \frac{1}{\eta}\ln\sum_{k=1}^{u}e^{\eta r^{(T+1)}(k)}\\ &\ge \frac{1}{\eta}\ln e^{\eta r^{(T+1)}(k^*)}=r^{(T+1)}(k^*)=-\mathcal{S}_T^{min},\end{aligned} \tag{6}$$

where k^* is the index of the collector with the highest reputation (i.e., the collector that behaves best). Combining (5) and (6) im-mediately obtains $\mathcal{L}_T-\mathcal{S}_T^{min}\le\frac{\ln u}{\eta}+\frac{\eta T}{2}$. This completes our proof. □

Note that for any T, we can specify $\eta=\sqrt{\frac{\ln u}{T}}$ and get $\mathcal{L}_T-\mathcal{S}_T^{\min}\le\frac{3}{2}\sqrt{T\ln u}$. We can double the value of T and reset η every time T transactions have been provided by provider p. As a result, after $T_{total}=T+2T+\cdots+2^{h-1}T=(2^h-1)T$ transactions, we have

$$\begin{aligned}&\mathcal{L}_{T_{total}}-\mathcal{S}_{T_{total}}^{\min}\\ &\le \frac{3}{2}\sqrt{T\ln u}+\frac{3}{2}\sqrt{2T\ln u}+\cdots+\frac{3}{2}\sqrt{2^{h-1}T\ln u}\\ &= \frac{3}{2}\sqrt{T\ln u}\sum_{k=0}^{h-1}(\sqrt{2})^k=\frac{3}{2(\sqrt{2}-1)}\sqrt{T\ln u}\Big((\sqrt{2})^h-1\Big)\\ &\le \frac{3}{2(\sqrt{2}-1)}\sqrt{T\ln u}\sqrt{2^h-1}=\frac{3\sqrt{\ln u}}{2(\sqrt{2}-1)}\sqrt{(2^h-1)T}\\ &= O\Big(\sqrt{(2^h-1)T}\Big)=O\Big(\sqrt{T_{total}}\Big).\end{aligned}$$

As h can be arbitrarily large, we thus obtain the $O(\sqrt{T_{total}})$ bound of loss for an arbitrarily large T_{total}.

4.3 Liveness Analysis

As dishonest collectors may wrongly label a transaction, some valid transactions may be delivered to governors with an "invalid" label and thus further ignored. Note that our protocol allows repeated uploading of a transaction until it appears on the chain. Here we analyze the delay for any valid transaction to be included in a block and show the liveness of our protocol. Our main theorem is as follows.

Theorem 2. *Consider a provider p connected to a group of u collectors, $u_0\ge 1$ of which are honest. Assume that p repeatedly uploads a valid transaction until it is verified and packed into a block by a governor. For any valid transaction tx provided by p, let $T_{repeated}$ denote the number of times that tx is uploaded by p, and $T_{repeated}=\infty$ if tx is never verified. Then for any integer $d>0$,*

$$\Pr[T_{repeated}\ge d]\le(1-u_0/u)^{d-1}.$$

Proof. An important observation is that any honest collector linked to p will never face a decline in reputation. Meanwhile, any detection of dishonest behavior will decrease the collector's reputation. In our mechanism, while processing the t-th transaction, a collector with reputation $r^{(t)}$ is chosen with probability proportional to $\exp(\eta r^{(t)})$, which is an increasing function of $r^{(t)}$. Therefore, we have the following claim. □

Claim 1. *Let $1\le u_0\le u$ be the number of honest collectors connected with p. Then for any valid transaction tx provided by p, the probability that tx is verified by governor g is no less than u_0/u.*

Further, notice that governor g chooses a collector based only on reputation, which is independent when processing different transactions. Thus the theorem is proved.

As a result, we have the following corollary.

Corollary 1. *Under the condition of Theorem 2, we have*

$$\mathbb{E}[T_{repeated}] \leq u/u_0.$$

For a given valid transaction tx, a dishonest collector may choose to mislabel only tx (while labeling all other transactions correctly). This would cause no decline in its reputation. Therefore, for tx, the probability that an honest collector is chosen remains u_0/u.

Meanwhile, the assumption that at least one honest collector is connected to any provider is reasonable. In reality, a provider would select to connect with reliable collectors. However, if all of the collectors linked to a provider are dishonest, then this provider is always blocked.

4.4 Incentive Analysis

A considerable benefit of introducing reputation into our protocol is the provision of incentives for collectors to behave honestly. In general, as participants in a permissioned system have sufficient computational resources to complete a given task, a collector's reputation should be a good measure of its reliability and honesty.

The preceding sections demonstrate the safety and liveness of our protocol. Here we show that our mechanism provides incentives for collectors to behave honestly, thus enhancing the efficiency of executing the protocol. Typically, to damage the efficiency of our protocol, a collector can act in two ways: it can misreport the correct status (*valid* or *invalid*) of a transaction or it can forge a transaction and report it to the governors. The following theorem concerns the first case.

Theorem 3. *Consider a provider p and its connected collector set $\{c(1), \cdots, c(u)\}$. Assume a transaction tx is provided to some dishonest collector $c(k)$ that mislabels tx and sends it to governors. Let $\Delta V(k)$ be the profit change of $c(k)$ by misreporting the status of the transaction compared with correctly labeling it. If there exists an honest collector $c(k_0)$, then we have:*

$$\mathbb{E}[\Delta V(k)] < 0.$$

Proof. Let $r^{(t)}(k)$ for $\forall k \in \{1, 2, ..., u\}$ be the reputation of collector $c(k)$ before dealing with the t-th transaction. Assume that at the end of the current slot, the total profit associated with provider p is F, then the revenue of $c(k)$ is

$$V(k) = \frac{\exp\big(\eta r^{(T+1)}(k)\big)}{\sum_{j=1}^{u} \exp(\eta r^{(T+1)}(j))} \cdot F.$$

where T is the total number of transactions in the current slot.

If $c(k)$ misreports the t-th transaction tx, then either of the following situations happens:

(1) $c(k)$ labels valid tx as "invalid". In this case, one honest collector, denoted by $c(k_0)$, correctly labels the transaction, and the governor will check the transaction with probability at least

$$\frac{\exp\big(\eta r^{(t)}(k_0)\big)}{\sum_{j=1}^{u} \exp(\eta r^{(t)}(j))} > 0.$$

(2) $c(k)$ labels invalid tx as "valid", and the governor will check the transaction with probability at least

$$\frac{\exp\big(\eta r^{(t)}(k)\big)}{\sum_{j=1}^{u} \exp(\eta r^{(t)}(j))} > 0.$$

If tx is not checked in our counterfactual situation, then the revenue $\widehat{V}(k)$ of $c(k)$ remains $V(k)$. However, as long as tx is checked, which always happens with positive probability, $c(k)$ loses reputation of 1, and the revenue of $c(k)$ becomes

$$\widehat{V}(k) = \frac{\exp\big(\eta\big(r^{(T+1)}(k) - 1\big)\big)}{\exp(\eta(r^{(T+1)}(k) - 1)) + \sum_{j \neq k} \exp(\eta r^{(T+1)}(j))} \cdot F.$$

As we have:

$$\frac{\exp\big(\eta r^{(T+1)}(k)\big)}{\sum_{j=1}^{u} \exp(\eta r^{(T+1)}(j))} > \frac{\exp\big(\eta\big(r^{(T+1)}(k) - 1\big)\big)}{\exp(\eta(r^{(T+1)}(k) - 1)) + \sum_{j \neq k} \exp(\eta r^{(T+1)}(j))},$$

and as a result, $\mathbb{E}[\Delta V(k)] = \mathbb{E}\Big[\widehat{V}(k) - V(k)\Big] < 0.$ □

Any dishonest collector attempting to forge a transaction will encounter the security of the digital signature scheme. It can fake a never-occurring transaction with only the negligible probability of security parameter λ. Considering both cases of malicious behavior, our softmax reputation algorithm provides a strong incentive for collectors to behave honestly.

5 Simulation

In this section, we further conduct simulations to evaluate the real-life performance of our protocol.

5.1 Setup

We first simulate a single slot with $T = 10,000$. The simulation includes $l = |P| = 300$ providers and $n = |C| = 200$ collectors. The number of governors $m = |G|$ need not be specified, because it does not affect the execution of the protocol. As all governors are alike, we suppose that there is only *one* governor for brevity. This single governor is linked with all collectors. For each provider, the simulation continues until $T = 10,000$ of its uploaded transactions have been verified.

In this simulation, each transaction uploaded by a provider is independently set as valid with a probability of 0.5. For each provider, the simulation continues until $T = 10,000$ of his uploaded transactions have been verified. We suppose that if a valid transaction uploaded by some provider is contained in $UncheckedList$, the provider will repeatedly upload the transaction until it appears on the chain. (Section 4.3 has demonstrated the finitude of this process.) However, any invalid transaction that is contained in $UncheckedList$ or $InvalidList$ rejected will be abandoned by the provider.

We consider three cases, each with a different number of dishonest collectors, denoted by dc: 40, 80, and 120,

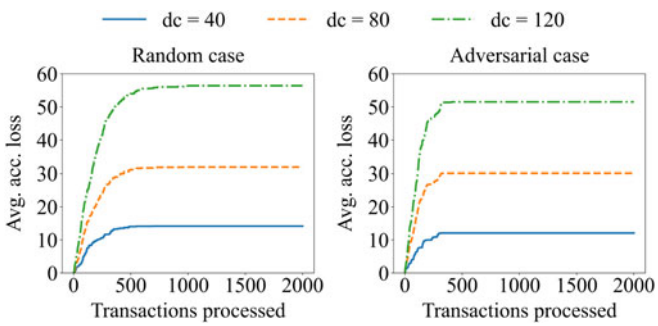


Fig. 4. Accumulated verification loss for all verified transactions in a single slot. The value of loss is averaged over all providers.

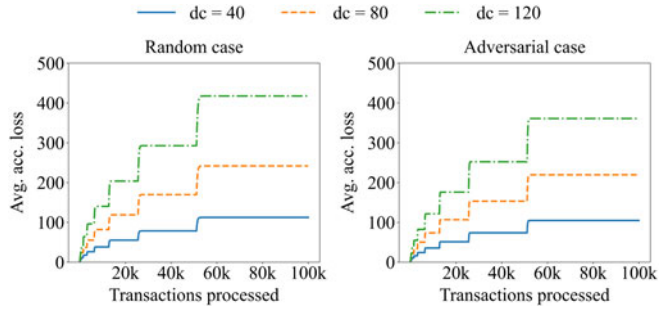


Fig. 5. Accumulated verification loss for all verified transactions in the whole process. The value of loss is averaged over all providers.

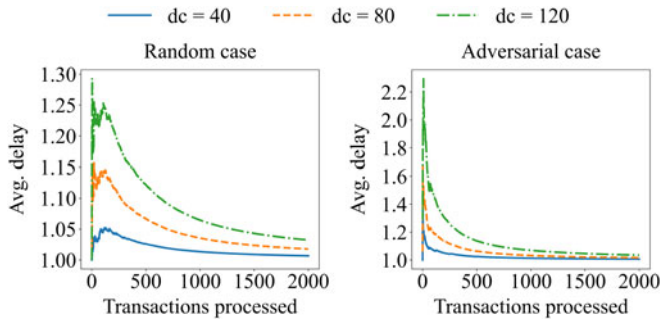


Fig. 6. Average delay for a provider to upload a transaction in a single slot.

representing 20%, 40%, and 60% of all collectors, respectively. (The remaining $n - dc$ collectors are always honest.) We suppose that each provider uniformly chooses at random to connect with $u = 10$ collectors, under the limitation that each provider links with *at least one* honest collector. Such an assumption matches with the theoretical analysis in Section 4.3. The parameter η is set as $\eta = \sqrt{\ln u/T}$.

We consider two kinds of behavior for dishonest collectors: (1) always randomly label a transaction with either label's probability 0.5, which is called as "random case", and (2) always wrongly label a transaction, which we call as "adversarial case". The random case corresponds to a dishonest collector being lazy and never checking transactions, and the adversarial case represents dishonest collectors as malicious troublemakers in the system.

In this simulation, we use the following metrics to evaluate our protocol.

- *Average accumulated verification loss:* the accumulated verification loss in all verified transactions averaged over all providers. This reflects the efficiency of our protocol.
- *Average delay of transactions:* the average delay for a provider to upload a transaction. This reflects the liveness of our protocol. Besides the valid transactions, we also consider those verified invalid transactions when calculating the average delay, as we focus on the overall behavior of our algorithm on all transactions.
- *Average exponential reputation of dishonest collectors:* the average of $\exp(\eta r)$ for all dishonest collectors, where r is the collector's reputation. Section 3 shows that a collector's reward is proportional to $\exp(\eta r)$. This metric reflects the incentive brought by our protocol.

We also simulate a whole process containing multiple slots. The first slot includes 100 transactions and the last slot includes 51,200 transactions. Totally there are 102,300 transactions in the whole process. We only show the result of average accumulated verification loss because the other two metrics are not cumulative, which means it's just a simple independent repetition to consider them in multiple slots.

5.2 Results

Fig. 4 illustrates the accumulated verification loss of the governor averaged over all providers in a single slot. For clarity, we only plot the first 2,000 transactions. The loss does not increase after about 1,000 transactions in all cases. For the $T = 10,000$ transactions verified in total, no more than 0.6% are invalid, even when 60% of collectors are dishonest. With only 20% of collectors dishonest, this ratio decreases to about 0.1%. The verification loss is smaller in the adversarial case than in the random case, as the dishonest collectors lose reputation more rapidly, thus having less probability of being chosen.

Fig. 5 depicts the accumulated verification loss averaged over all providers in the whole process. In each slot, the loss only increases at the beginning. The ratio of loss is less than 0.5% over the $T = 102,300$ transactions verified in total, even in the worst case that 60% of collectors are dishonest.

There is a discrepancy between the theoretical bound in Theorem 1 and the loss in our simulation. This is because many providers are connected to more than one honest collector in the simulation. While the extreme case considered in Theorem 1 has exactly one honest collector connected to the provider.

Fig. 6 reflects the average delay for providers to upload a transaction. Similar to Fig. 4, we only plot the first 2,000 transactions, as the delay of each subsequent transaction is 1 in all cases. Even when adversarial collectors are more likely to be chosen (as at the start of the 60% adversarial dc case), the average delay for any transaction does not exceed 4. In the adversarial case, the average delay decreases more rapidly than in the random case, as the dishonest collectors are quickly excluded from subsequent transactions. The simulated liveness performance is much better than the theoretical bound in Theorem 2.

Fig. 7 depicts the dishonest collectors' average exponential reputation. Within 2,500 transactions, the probability that a dishonest collector is chosen is no more than 6×10^{-8} in the random case, and no more than 4×10^{-16} in the adversarial case. This shows that as the number of

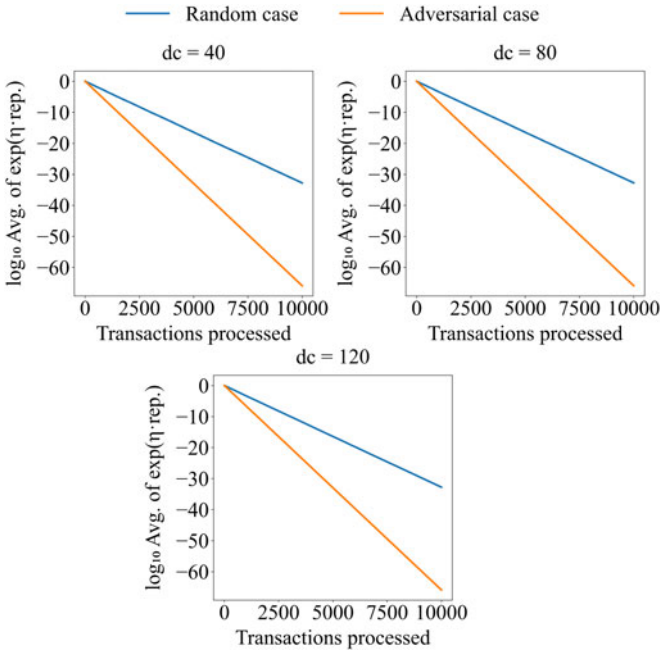


Fig. 7. Average $\exp(\eta r)$ for all dishonest collectors in a single slot, where r is a collector's reputation.

processed transactions accumulates, the probability of selecting a dishonest collector decreases exponentially. The dishonest collectors' rewards also plummet exponentially. The fraction of dishonest collectors has almost no impact on this consequence. Therefore, our mechanism provides a huge incentive for collectors to behave well.

There is an essential connection between dishonest collectors' reputations and the simulation results in Figs. 4 and 6. The average exponential reputation of all dishonest collectors indicates the probability of selecting a dishonest collector. Therefore, once their reputation has decreased to an extremely low level, there is little chance for dishonest collectors to harm the efficiency and liveness of our protocol.

In all, our simulation demonstrates that our protocol owns the features of high efficiency, firm liveness, and promising incentives.

6 Applications

The key objective of our protocol is to reduce costs associated with verifying invalid transactions. It therefore facilitates the selection of reliable collectors to increase the efficiency of collecting valid transactions. This section demonstrates the practical application of our model to data collection in the Internet of Things (IoT) and second-hand markets.

6.1 IoT Data Collection

A direct application of our mechanism is to the IoT, in which cloud nodes use data received from sensors and devices to make decisions and then return instructions. However, limits of processing capacity and varying network interfaces adopted by different sensors and devices prevent them from sending data to cloud nodes directly. Some of the sent data may even be wrong due to unpredictable faults. Therefore, gateway nodes play an important role in aggregating data from data providers and uploading filtered data to cloud nodes.

Our system mitigates cloud nodes' losses when gateway nodes attach wrong labels to raw data from end equipment. The global reputation maintained for each gateway node ensures that a label proposed by an honest gateway node will be adopted with a higher probability. This improves the efficiency of screening data. Furthermore, our incentive mechanism promotes honesty among gateway nodes.

6.2 Second-hand Markets

Our reputation mechanism can mitigate the problem of asymmetric information in a transaction. In a situation where collectors can access information much more easily than governors, our protocol helps to identify honest collectors. As an example, we demonstrate that our system suits the classic problem of second-hand markets, or the market for "lemons" [23].

Consider multiple trading platforms simultaneously receiving information about a product from its owner. To gain a better return, the owner may deliberately exaggerate its quality, and it is the platforms' responsibility to check the status of the offered product. However, in reality, a trading platform may neglect the buyer's interest or even collude with the seller in return for a higher profit, thus harming the whole market.

Our model suits this scenario: sellers are providers, trading platforms are collectors, and buyers/market regulators are governors. Here, our protocol guarantees that the buyer's or market regulator's loss is bounded by the square root of the total amount of transactions. Meanwhile, any trading platform behaving dishonestly will lose reputation and rapidly lose buyers' trust. As a consequence, our mechanism facilitates the fairness of second-hand markets.

7 Conclusion

This work proposes a permissioned blockchain model for a hierarchical system with three types of participant: providers, collectors, and governors. Providers forward transactions to collectors; collectors label received transactions after checking them, then broadcast them to governors; and governors validate some of the labeled transactions, pack valid transactions into a block, and append the block to the ledger. Governors face difficulties in judging transactions, which stem from the varying opinions of collectors and deficient incentives for collectors to label transactions truthfully. To overcome these problems, we design a reputation-based protocol for the collection, verification, and packing of transactions in a hierarchical permissioned environment. Here, reputation is a measure of the reliability of collectors. Theoretically and empirically, our protocol is shown to have high efficiency, firm liveness, and strong incentives. Furthermore, we explicitly demonstrate the applications of our solution to IoT data collection and second-hand markets.

In our softmax reputation-based protocol, each governor computes the reputations of the collectors corresponding to each provider–collector pair. To be specific, consider a provider p_i connected with u_i collectors $C_i = \{c_i(1), c_i(2), \ldots, c_i(u_i)\}$. Then, for each p_i, $i = 1, \ldots, l$, the governors shall store the reputations of all collectors in C_i in our protocol.

Therefore, if there are l providers and n collectors, then the total storage cost of each governor is $O(ln)$. The governors incur high storage costs, as they need detailed reputation information to enhance the accuracy of collector selections. Therefore, future work will consider the crucial and difficult challenge of compressing the storage of reputations to save space while maintaining performance accuracy.

Acknowledgments

Hongyin Chen, Zhaohua Chen, and Hongyi Ling are joint first authors.

References

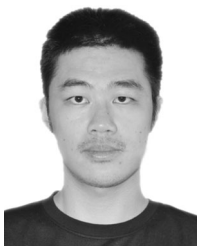
[1] E. Androulaki *et al.*, "Hyperledger fabric: A distributed operating system for permissioned blockchains," in *Proc. 30th EuroSys Conf.*, 2018, pp. 30:1–30:15.
[2] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," Manubot, 2008. Accessed: Nov. 27, 2021. [Online]. Available: https://goo.gl/MqyCW7
[3] V. Buterin, "On public and private blockchains." Accessed: Nov. 27, 2021. [Online]. Available: https://blog.ethereum.org/2015/08/07/on-public-and-private-blockchains/
[4] G. Wood *et al.*, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, vol. 151, no. 2014, pp. 1–32, 2014.
[5] Litecoin, "Litecoin github," 2021. Accessed: Nov. 27, 2021. [Online]. Available: https://litecoin.org/
[6] C. Cachin and M. Vukolic, "Blockchain consensus protocols in the wild," *CoRR*, vol. abs/1707.01873, 2017. Accessed: Nov. 27, 2021. [Online]. Available: http://arxiv.org/abs/1707.01873
[7] M. Castro and B. Liskov, "Practical byzantine fault tolerance and proactive recovery," *ACM Trans. Comput. Syst.*, vol. 20, no. 4, pp. 398–461, 2002.
[8] M. Gupta, P. Judge, and M. H. Ammar, "A reputation system for peer-to-peer networks," in *Proc. 13th Int. Workshop Netw. Oper. Syst. Support Digit. Audio Video*, 2003, pp. 144–152.
[9] R. Zhou and K. Hwang, "Powertrust: A robust and scalable reputation system for trusted peer-to-peer computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 18, no. 4, pp. 460–473, Apr. 2007.
[10] S. Buchegger and J.-Y. Le Boudec , "A robust reputation system for mobile ad-hoc networks," in *Proc. Second Workshop Econ. P2P Syst.*, 2004.
[11] K. J. Hoffman, D. Zage, and C. Nita-Rotaru , "A survey of attack and defense techniques for reputation systems," *ACM Comput. Surv.*, vol. 42, no. 1, pp. 1:1–1:31, 2009.
[12] M. Kinateder and S. Pearson, "A privacy-enhanced peer-to-peer reputation system," in *Proc. 4th Int. Conf. Electron. Commerce Web Tech.*, 2003, pp. 206–215.
[13] Y. Wang, Z. Cai, G. Yin, Y. Gao, X. Tong, and G. Wu, "An incentive mechanism with privacy protection in mobile crowdsourcing systems," *Comput. Netw.*, vol. 102, pp. 157–171, 2016.
[14] R. Dennis and G. Owen, "Rep on the block: A next generation reputation system based on the blockchain," in *Proc. 10th Int. Conf. Internet Tech. Secured Trans.*, 2015, pp. 131–138.
[15] R. Dennis and G. Owenson, "Rep on the roll: a peer to peer reputation system based on a rolling blockchain," *Int. J. Digit. Soc.*, vol. 7, no. 1, pp. 1123–1134, 2016.
[16] A. Schaub, R. Bazin, O. Hasan, and L. Brunie, "A trustless privacy-preserving reputation system," in *Proc. 31st IFIP TC 11 Int. Conf. ICT Syst. Secur. Privacy Protection* , 2016, pp. 398–411.
[17] F. Gai, B. Wang, W. Deng, and W. Peng, "Proof of reputation: A reputation-based consensus protocol for peer-to-peer network," in *Proc. 23ed Int. Conf. Database Syst. Adv. Appl.*, 2018, pp. 666–681.
[18] M. Nojoumian, A. Golchubian, L. Njilla, K. Kwiat, and C. Kamhoua, "Incentivizing blockchain miners to avoid dishonest mining strategies by a reputation-based paradigm," in *Proc. Sci. Inf. Conf.*, 2018, pp. 1118–1134.
[19] M. Zhang, J. Li, Z. Chen, H. Chen, and X. Deng, "Cycledger: A scalable and secure parallel protocol for distributed ledger via sharding," in *Proc. 36th IEEE Int. Parallel Distrib. Process. Symp.*, 2020, pp. 358–367.
[20] C. Cachin, R. Guerraoui, and L. Rodrigues, *Introduction to Reliable and Secure Distributed Programming*, 2nd ed. Berlin, Germany: Springer, 2011.
[21] S. Arora, E. Hazan, and S. Kale, "The multiplicative weights update method: A meta-algorithm and applications," *Theory Comput.*, vol. 8, no. 1, pp. 121–164, 2012.
[22] S. Micali, M. O. Rabin, and S. P. Vadhan, "Verifiable random functions," in *Proc. 40th Annu. Symp. Found. Comput. Sci.*, 1999, pp. 120–130.
[23] G. A. Akerlof, "The market for lemons: Quality uncertainty and the market mechanism," in *Uncertainty in Economics*. Cambridge, MA, USA: Academic, 1978, pp. 235–251.

Hongyin Chen received the BS degree from the School of Electrical Engineering and Computer Science, Peking University, in 2020. He is currently working toward the doctorate degree with the Center on Frontiers of Computing Studies, Peking University. His current research interests include blockchain and mechanism design.

Zhaohua Chen received the BS degree from the School of Electrical Engineering and Computer Science, Peking University, in 2021. He is currently working toward the PhD degree with the Center on Frontiers of Computing Studies, Peking University. His research interest include various subjects of theoretical computer science.

Yukun Cheng received the PhD degree from Shanghai University, in 2010, and completed postdoctoral studies with the College of Computer Science and Technology, Zhejiang University from 2010 to 2013. She is a professor with the Suzhou University of Science and Technology. Her current research interests include mechanism design problems in algorithmic game theory, internet market design, and combinatorial optimization.

Xiaotie Deng (Fellow, IEEE) received the BSc degree from Tsinghua University, the MSc degree from the Chinese Academy of Sciences, and the PhD degree from Stanford University, in 1989. He is currently a chair professor with Peking University. He has taught at Shanghai Jiaotong University, the University of Liverpool, City University of Hong Kong, and York University. Before that, he was an NSERC International fellow with Simon Fraser University. His current research interests include on algorithmic game theory, with applications to Internet economics and finance. His works cover online algorithms, parallel algorithms, and combinatorial optimization. He is an ACM fellow for his contribution to the interface of algorithms and game theory and an IEEE fellow for his contributions to computing in partial information and interactive environments.



Wenhan Huang received the BS degree from Zhiyuan College, Shanghai Jiao Tong University. He is currently working toward the doctorate degree with the Department of Computer Science and Engineering, Shanghai Jiao Tong University, China. His current research interests include blockchains, mechanism design, game theory, and machine learning.

Jichen Li received the BS degree in computer science from the School of Electrical Engineering and Computer Science, Peking University, in 2020. He is currently working toward the doctorate degree with the Center on Frontiers of Computing Studies, Peking University. His current research interests include blockchain and mechanism design.

Hongyi Ling received the BS degree from the School of Electrical Engineering and Computer Science, Peking University, in 2021. He is currently working toward the master's degree with ETH Zurich. His current research interests include algorithmic game theory and theoretical computer science.

Mengqian Zhang received the BE degree in computer science from the Ocean University of China, in 2018. She is currently working toward the doctorate degree with the Department of Computer Science and Engineering, Shanghai Jiao Tong University, China. Her current research interests include blockchains, algorithmic game theory, and mechanism design.